

System Map

Every page a customer can visit · every state a meal plan can hold · every automated thing happening behind the scenes — in plain English.

VERSION

dev

COMPILED

2026-05-06

If the version stamp above is older than your most recent commit, this PDF is stale — re-run `npm run render`.

Contents

01 What this document is

02 The big picture

03 All the pages

04 Common journeys

05 A meal plan's lifecycle

06 How signup works

07 What runs by itself

08 How the data fits together

09 Every banner, badge & modal

Appendix A Plain English ↔ code names

Appendix B Test accounts

1 What this document is

A single source you can read end-to-end to understand **how Dormers works as a product** — without grepping the codebase or asking the engineer.

This document is written for anyone who needs to *understand* the system: a new collaborator, a stakeholder reviewing the customer experience, a support agent learning what to expect when a customer says "my plan paused itself."

The diagrams use the names you'd see in the dashboard — **"Tonight's dish", "Skip my meal", "Pick a plan"** — not the internal variable names. Where the technical name actually matters (e.g. for the engineering team to implement against), Appendix A maps every customer-facing term to the matching code.

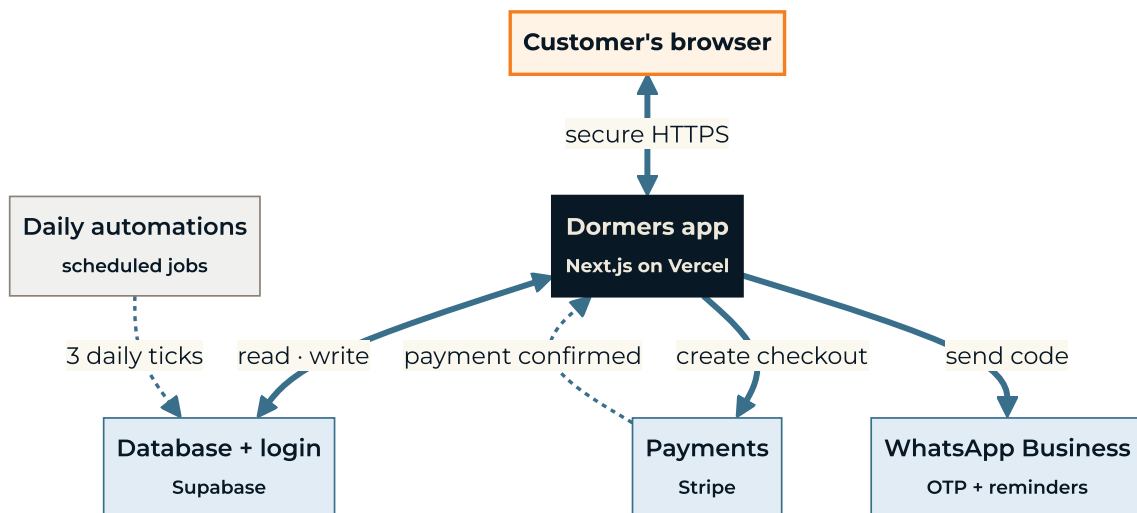
HOW THIS DOCUMENT STAYS CURRENT

Every section in this PDF was generated from the live source files in `src/app/` and `supabase/migrations/`. The version stamp on the cover is the git short-hash at build time — if you're reading this after newer code has shipped, regenerate the PDF.

2

The big picture

Dormers connects three customer-facing things to four behind-the-scenes services. The browser only ever talks to the Dormers app server; the Dormers app server fans the request out to the right service.



The customer never talks to Stripe, Supabase or WhatsApp directly. The Dormers app coordinates everything.

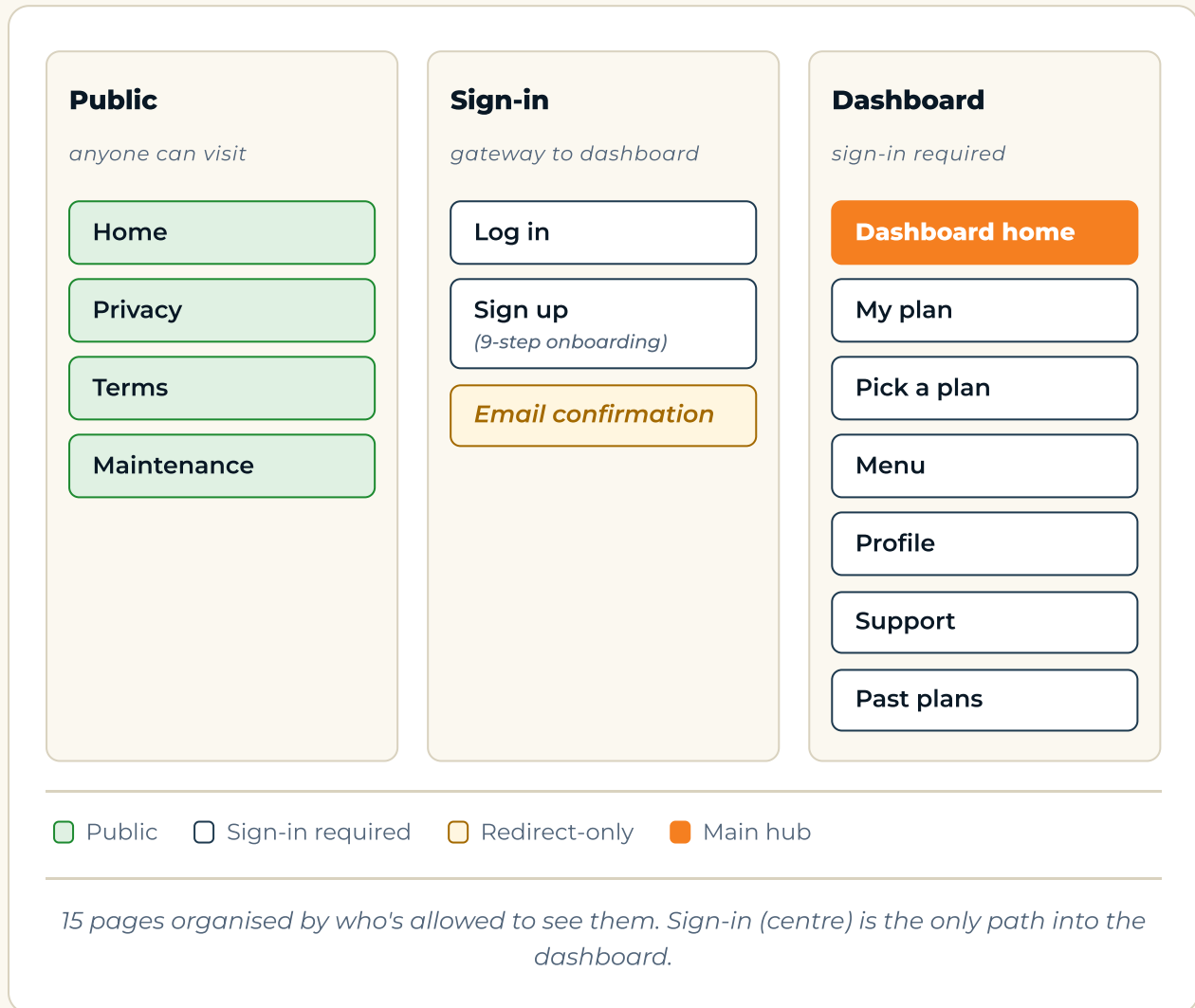
— What each piece does

- **Customer's browser** — what students see when they visit dormers.ae. Just shows pages and buttons.
- **Dormers app** — the brains. Decides what page to show, validates every action, and is the only piece allowed to talk to the other services.
- **Database + login** — stores customer profiles, their meal plans, their order history, and handles their email + password.
- **Payments** — handles credit-card collection and confirms when money arrives.
- **WhatsApp Business** — sends verification codes during signup and (separately) the daily "your meal is on its way" messages from operations.
- **Daily automations** — scheduled tasks that run by themselves to keep meal plans up to date, even when no customer is logged in. (See §7.)

3

All the pages

Dormers has 15 distinct pages plus a handful of behind-the-scenes endpoints. The pages split into three groups by who's allowed to see them.



— Page reference

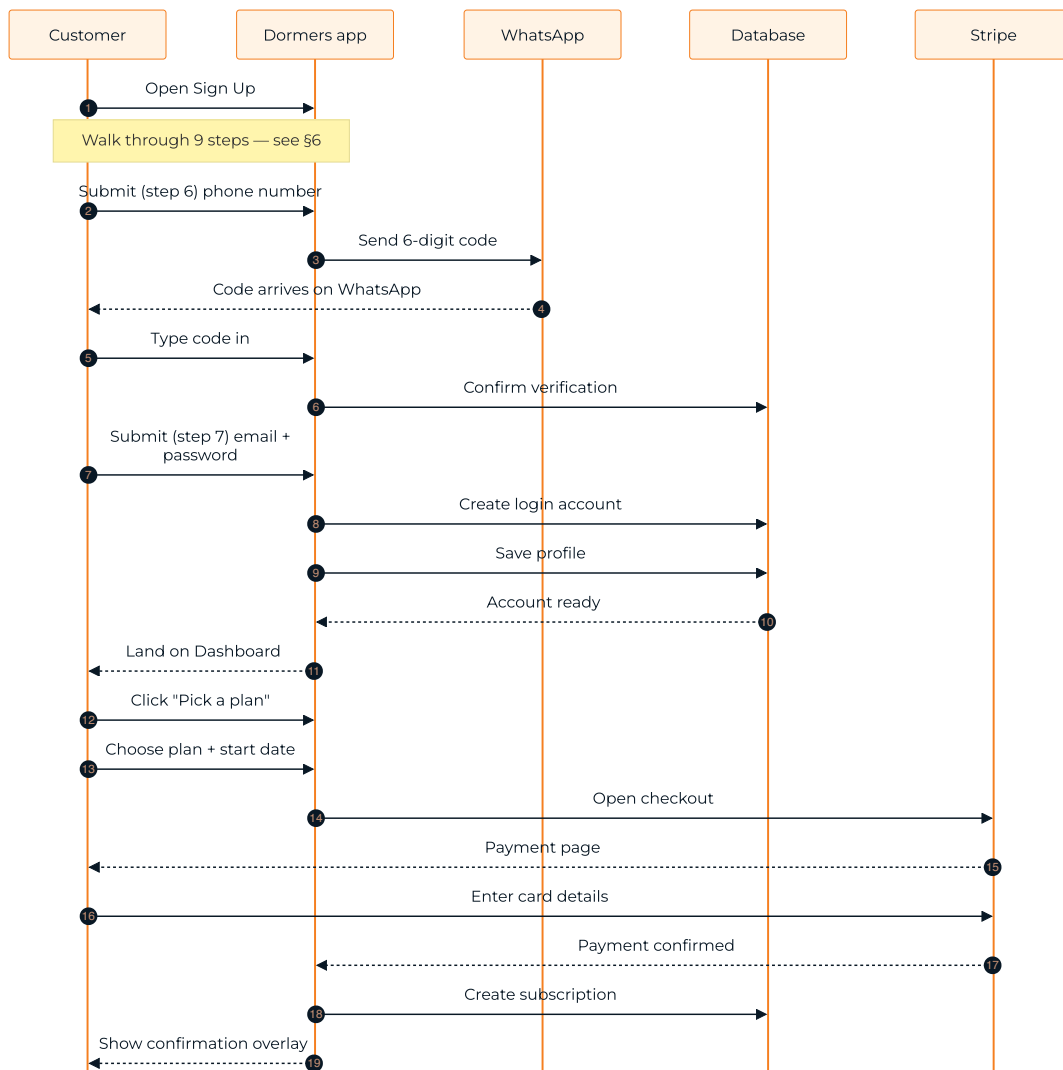
PAGE	WHO CAN SEE IT	WHAT'S THERE
Home	Anyone	Landing page, hero, sign-up CTA.
Privacy / Terms	Anyone	Static legal copy.
Maintenance	Anyone	Shown when the team takes the site offline.
Log in	Anyone	Email + password fields, "Forgot password?" path, OTP-based reset flow.
Sign up	Anyone	Multi-step onboarding (see §6) — gathers preferences, contact details, password.
Email confirmation	Anyone with a magic link	Activates email verification then bounces to the dashboard.
Dashboard home	Signed-in customers	Hero card with tonight's dish · skip / pause buttons · plan progress · stat row.
My plan	Signed-in customers	Current plan details · veg-day chips (if religious mix) · past plans grid.
Pick a plan	Signed-in customers	Pricing grid · checkout panel with date picker.
Menu	Signed-in customers	This week + next week of dishes, customised to the customer's preferences.
Profile	Signed-in customers	Edit details · security section (email · password · WhatsApp re-verify).
Support	Signed-in customers	FAQ + chatbot.
Past plans	Signed-in customers	Read-only list of every subscription that's ended.

4

Common journeys

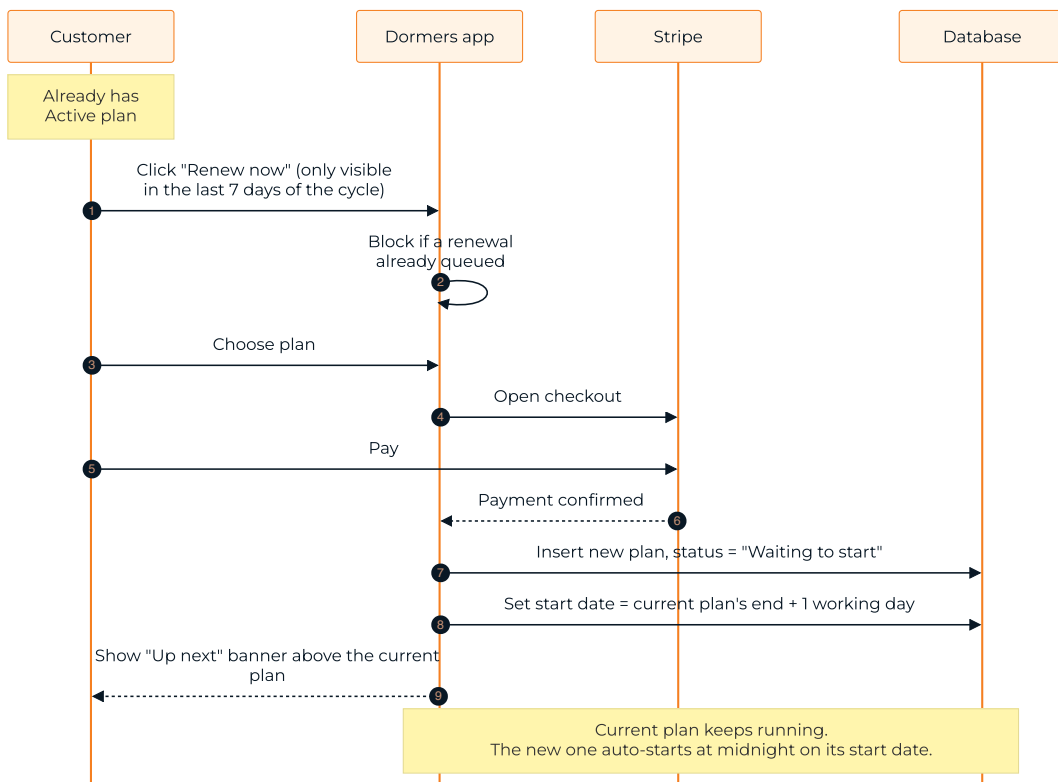
The seven flows below describe what happens — visually, step-by-step — when a customer does the most common things on Dormers.

— 4.1 Signing up & ordering the first plan



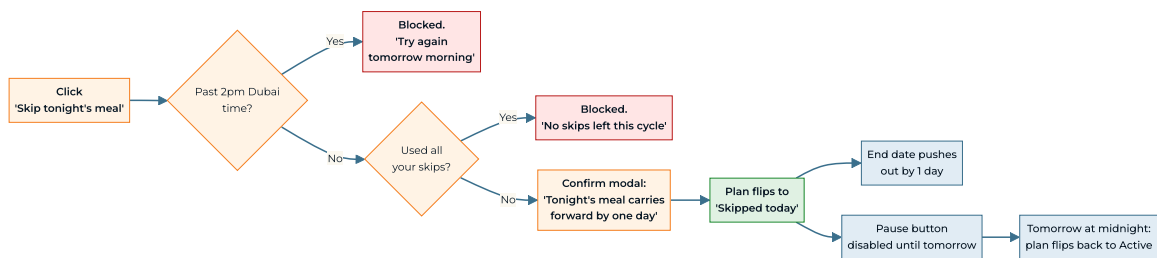
First-time signup → first paid plan, end-to-end.

— 4.2 Renewing — the next plan queues behind the current one



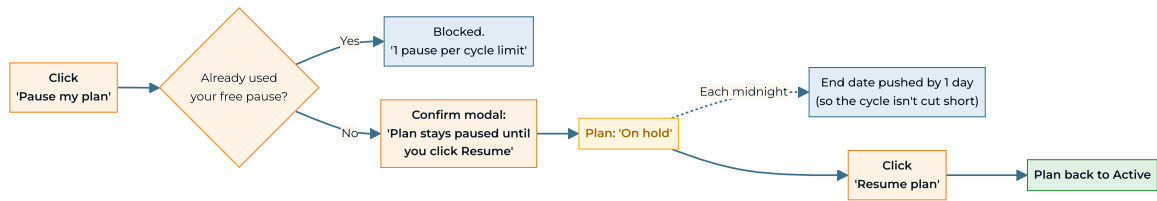
A renewal never replaces the current plan — it queues behind it. The current plan keeps delivering until it naturally ends.

— 4.3 Skipping today's meal



The skip flow. Skips don't lose meals — they shift the missed day to the end of the cycle.

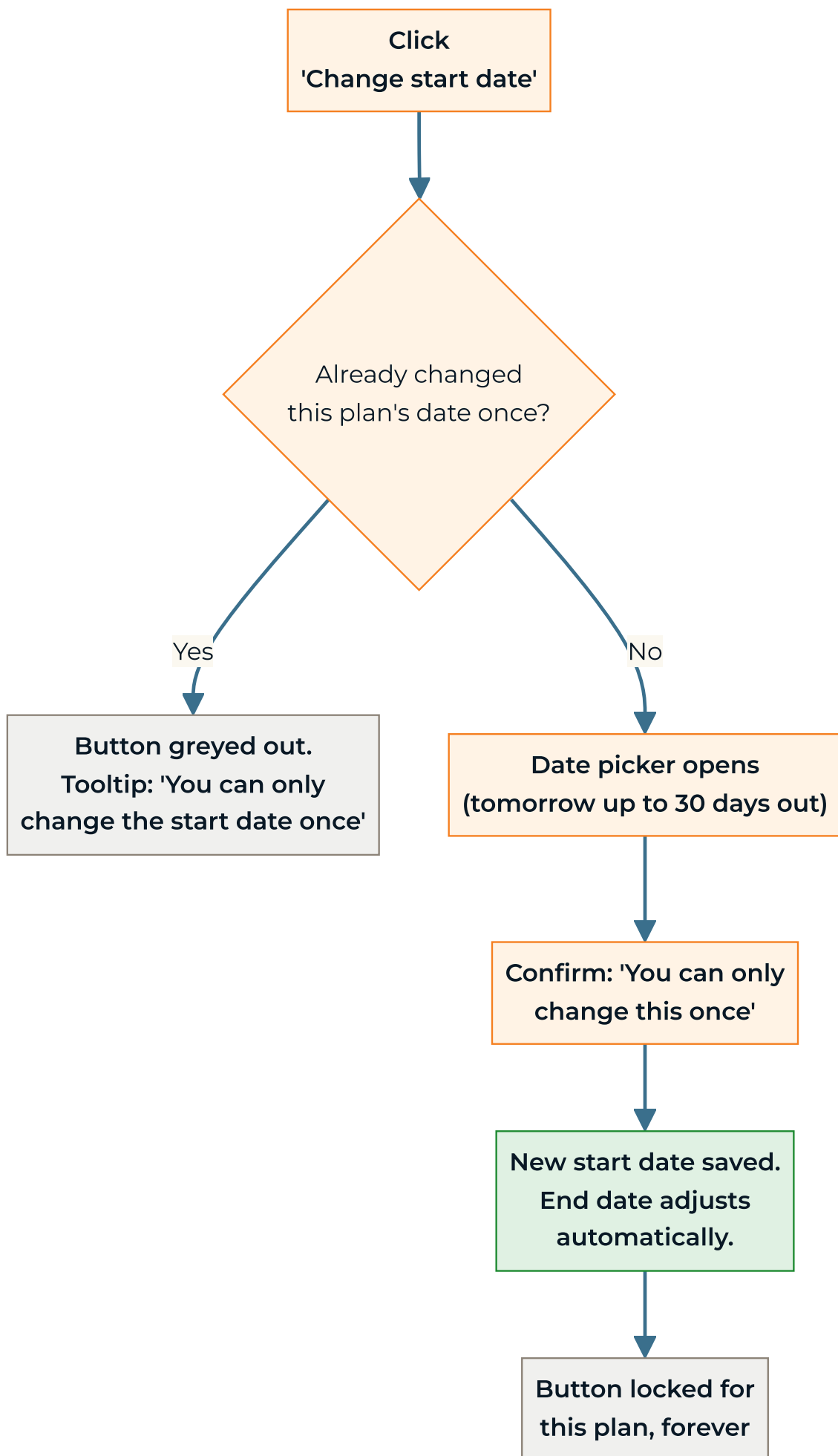
— 4.4 Pausing & resuming



Pausing freezes the clock. Each day the plan stays paused adds one day to the end date.

— 4.5 Changing the start date of a scheduled plan

Customers with a plan that hasn't started yet can shift its start date **once**. After that the button greys out for that plan.



One-shot date change per plan. Re-opens fresh at the next renewal.

— 4.6 Resetting your password

Three steps, all in-app. No magic-link emails to fish out of spam.

- 1 On the Log in page, click **"Forgot password?"** → enter email → Dormers emails a 6-digit code.
- 2 Type the code on screen → Dormers logs you into a temporary recovery session.
- 3 Set a new password → done. Stronger-than-default rules enforced.

— 4.7 Changing your email, password, or WhatsApp number

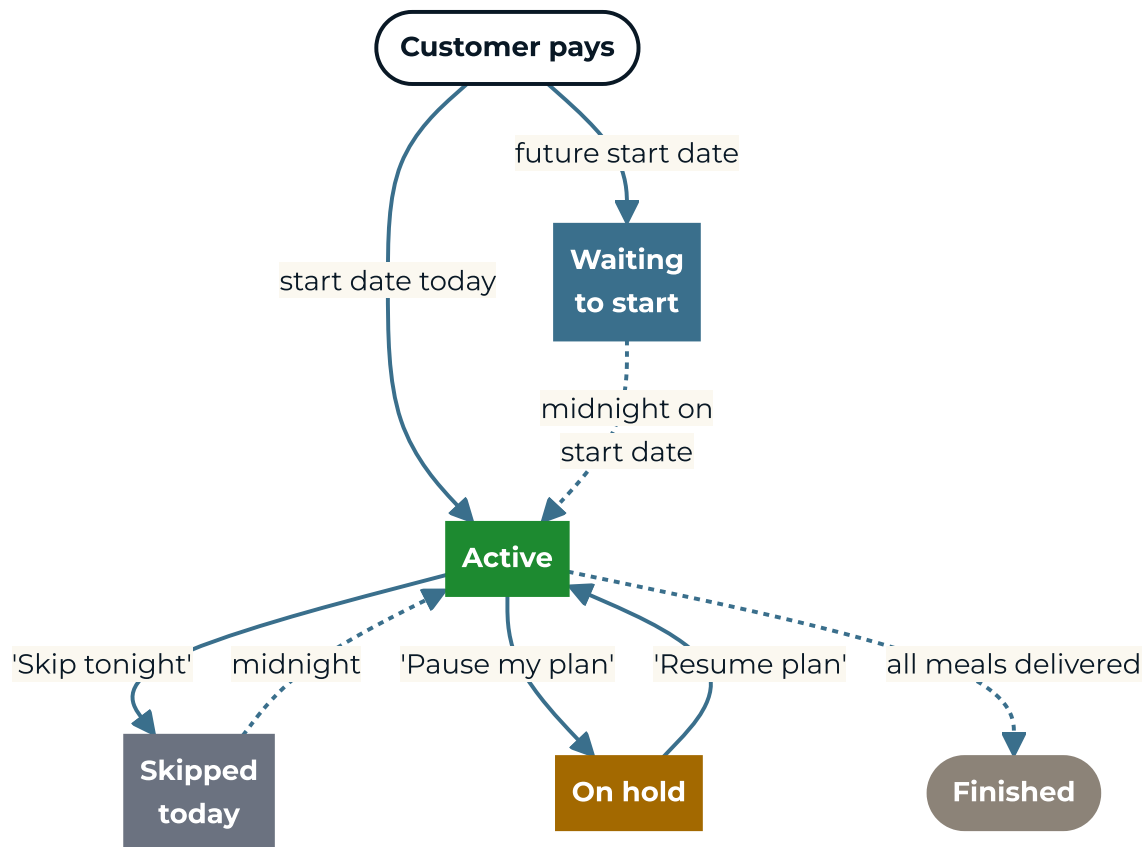
All three live in the **Security & verification** card on the Profile page.

- **Change email** — type new email; Dormers sends a confirmation link to the new address; the change takes effect when you click it.
- **Change password** — type current password (re-auth check), then the new one twice. Forgot path: "Email me a reset link" sends to the email on file.
- **Change & re-verify WhatsApp** — type new number → Dormers sends a 6-digit code on WhatsApp → enter code → number is saved as verified.

5

A meal plan's lifecycle

Every meal plan is in exactly one of five states at any moment. The diagram below shows every transition — what causes the plan to move from one state to the next.



The five states a meal plan can hold. Solid arrows are customer actions; dashed arrows happen automatically at midnight.

— What each state means

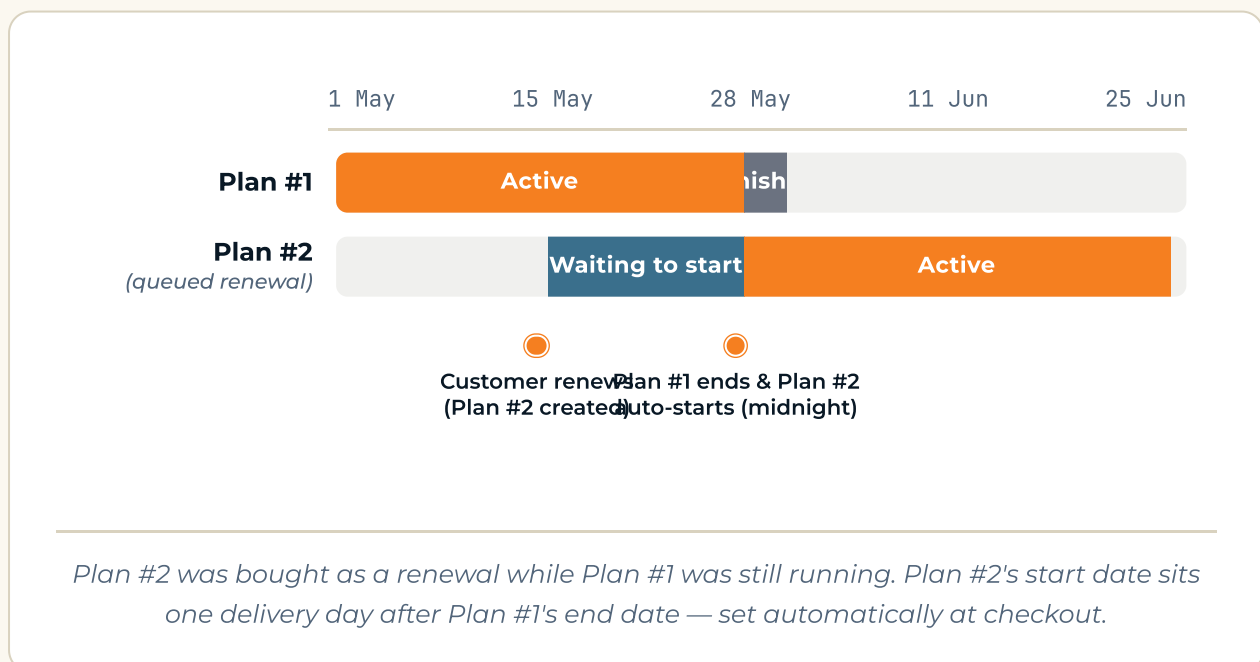
STATE	MEANING	CUSTOMER SEES
Waiting to start	Paid for, but the start date hasn't arrived yet.	"Starting Mon, 13 May" hero card.
Active	Currently delivering.	Tonight's dish, countdown timer, skip + pause buttons.
Skipped today	Active plan, but the customer skipped tonight. Auto-reverts to Active at midnight.	"You skipped today. Tomorrow's delivery is on track." Pause button disabled.
On hold	Customer hit pause. Stays paused until they hit resume — no time limit.	"Your plan is paused" hero. Big orange "Resume plan" button.
Finished	All meals delivered. The plan is in the customer's history.	Doesn't show on dashboard — appears in the "Past plans" page.

— The "one current + one queued" rule

A customer can hold up to two live plans at the same time:

- **One current plan** — the one delivering now (Active, On hold, or Skipped today).
- **One queued plan** — bought ahead of time as a renewal, status "Waiting to start".

The dashboard shows the current plan's progress in full. The queued one shows up as a single line above the hero: *"Up next: Monthly Premium · starts Mon 27 May"*. Trying to buy a third plan is blocked at checkout.



— How the end date is calculated

The end date isn't just *start date + 28 days*. It accounts for non-delivery days (Sundays for a 6-day-a-week plan; Saturdays + Sundays for a 5-day-a-week plan), the days the customer has paused, and the days they've skipped (each skip pushes the end out by 1 working day, so they don't lose the meal).

WORKED EXAMPLE

Monthly Premium · 6 days a week · starts Mon 29 Apr · 1 skip used → end date is

Tue 27 May (24 paid meals × 6-day weeks = 4 weeks of working days, plus the 4 Sundays in between, plus the 1 skip-extension day).

6

How signup works

Onboarding is nine logical steps. Two are conditional — only Religious Mix customers see the veg-days picker (step 3); the "Other" branches in dorm and university only appear if the customer picks "Other" instead of one of the listed options.

**1 · Meal preference
(Veg / Non-Veg / Religious Mix)**



**2 · Delivery week
(5 days or 6 days)**



Picked Religious Mix?

Yes



3 · Pick veg days

No



(max 4 of 5 or 5 of 6)

NO

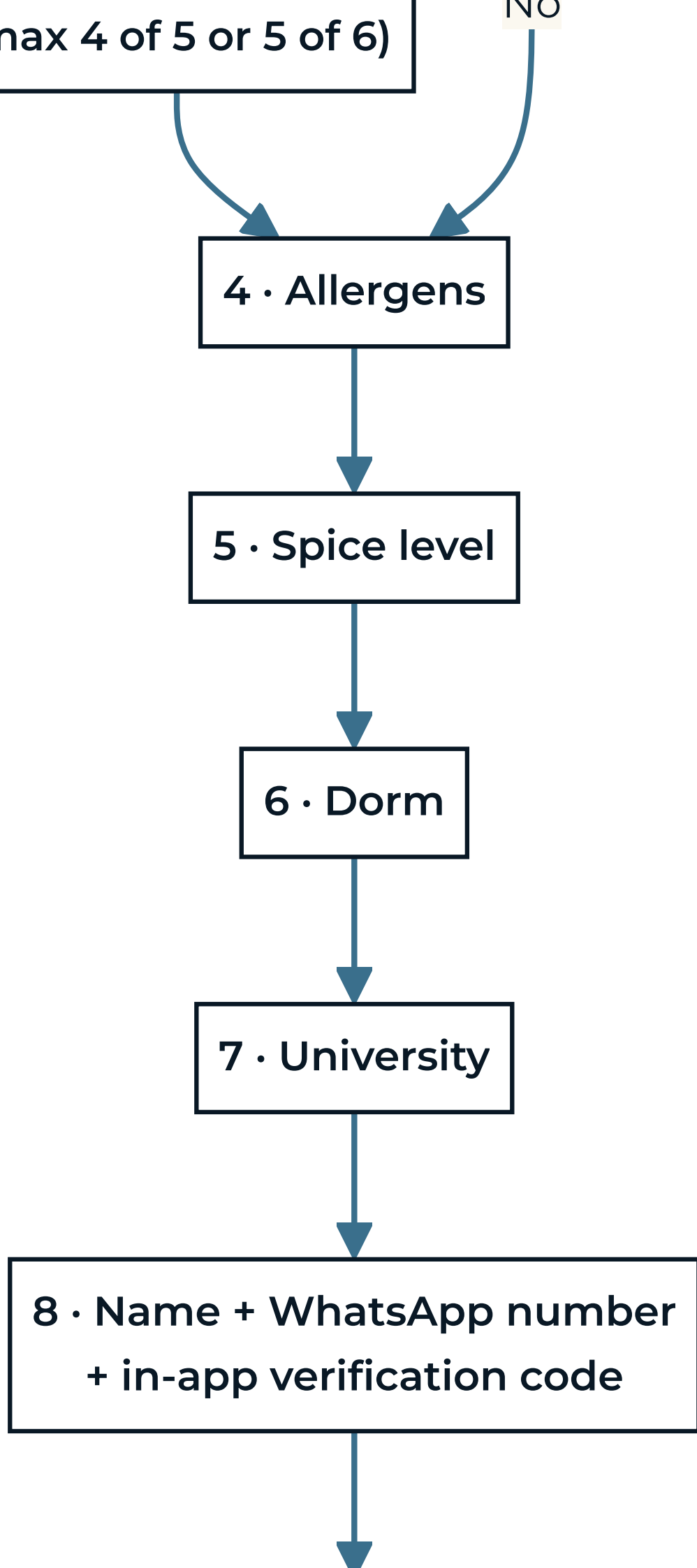
4 · Allergens

5 · Spice level

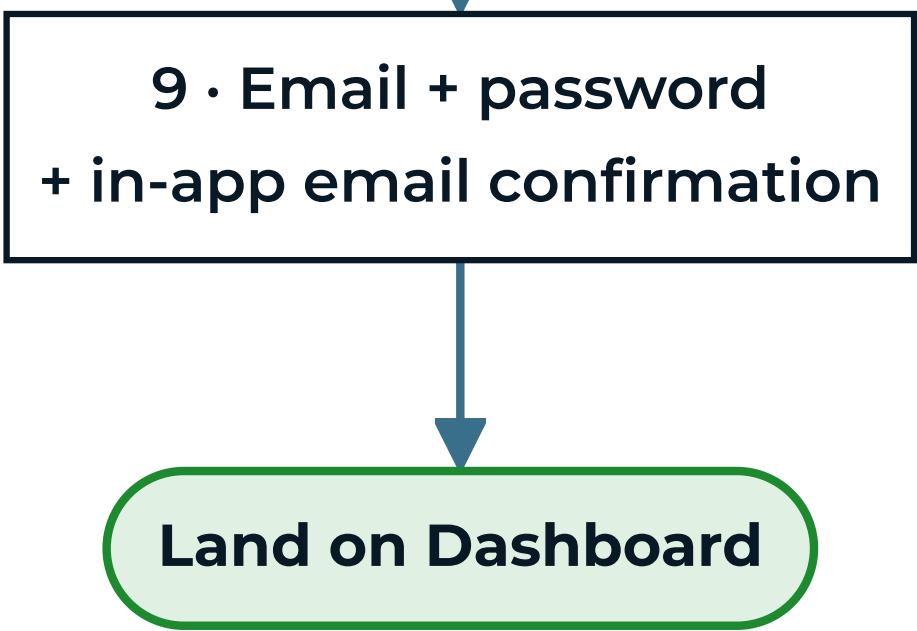
6 · Dorm

7 · University

8 · Name + WhatsApp number
+ in-app verification code



9 · Email + password
+ in-app email confirmation



Land on Dashboard

The nine onboarding steps. Religious Mix customers visit step 3; everyone else skips it.

— Why "Delivery week" comes *before* veg days

The veg-day picker on step 3 has to know how many working days there are in the customer's week so it can cap the picker correctly (you can pick at most 4 veg days if your week is 5 days; up to 5 if it's 6). Putting the week-type question first means the picker always knows how many options to offer.

— Server-side checks at the very end

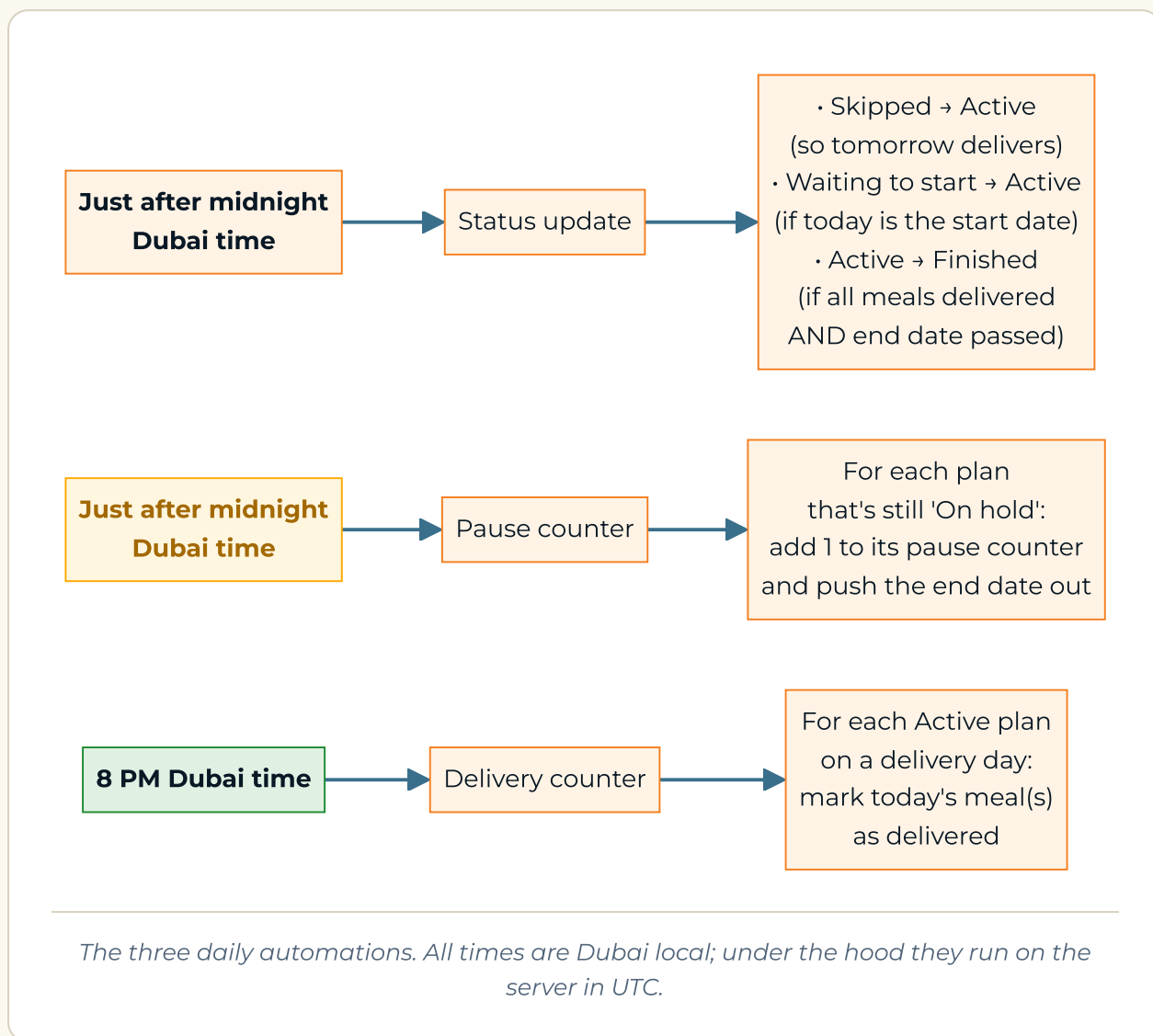
Right before the account is created, the server independently re-checks:

- Email looks like an email and isn't already taken
- Password is strong enough
- WhatsApp was actually verified within the last 30 minutes (UI flag is just a hint)
- For Religious Mix on a 5-day week: veg days picked are between 1 and 4

If any check fails, the customer sees a friendly error and stays on the same step.

7 What runs by itself

Three things happen automatically every day — no human has to push a button. They run on Dubai time so the customer's experience matches their wall clock.



— Why these run at these times

- **Status update at 00:05 Dubai** — runs *before* any deliveries that day, so a plan whose start date is today wakes up Active before 8 PM.
- **Pause counter at 00:10 Dubai** — same window. The 5-minute gap between this and Status update is just to keep them from interleaving.
- **Delivery counter at 20:00 Dubai** — when most of the kitchen's evening drops have happened. Counts every delivery that wasn't paused or skipped.

IMPORTANT: THE DELIVERY COUNTER IS TIME-BASED, NOT DELIVERY-EVENT-BASED

If a customer's meal didn't actually arrive (kitchen ops issue), the system *still* marks it delivered at 8 PM. Reconciliation happens out-of-band — operations adjusts manually if a delivery genuinely failed. This is a deliberate trade-off for simplicity.

— What happens when a customer clicks a button

Server-side, every action goes through one of these check-then-write flows:

BUTTON	SERVER CHECKS	THEN WRITES
Skip tonight's meal	Plan is Active · time is before 2 PM Dubai · skips remaining > 0	Status → Skipped today · skip counter +1 · end date pushed
Pause my plan	Plan supports pause (Premium / Max only) · hasn't paused before this cycle · isn't Finished	Status → On hold · timestamp the pause
Resume plan	Plan is currently On hold	Status → Active · clear pause timestamp
Change start date	Plan is Waiting to start · hasn't been changed yet · new date is tomorrow–30 days out	New start date · stamp the change · auto-recompute end date
Pick a plan / Renew	Profile complete · no plan already queued · trial cooldown OK · veg days valid	Open Stripe checkout

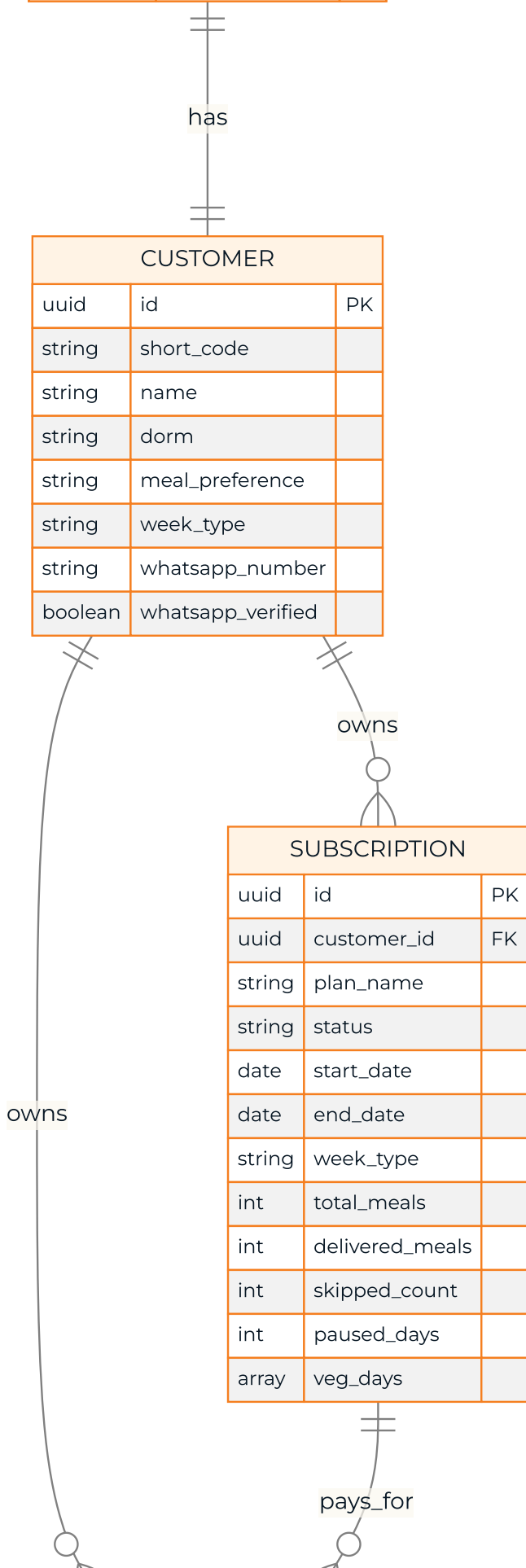
8

How the data fits together

Five tables hold everything Dormers knows about its customers. They link together as below — each line is a foreign key.

LOGIN		
uuid	id	PK
string	email	
timestamp	email_verified_at	

WHATSAPP_OTP		
uuid	id	PK
string	phone	
string	code_hash	
int	attempts	
timestamp	verified_at	



ORDER		
uuid	id	PK
uuid	customer_id	FK
uuid	subscription_id	FK
string	stripe_session_id	
decimal	price_per_meal	
int	meals_count	
timestamp	paid_at	

The five tables Dormers stores customer data in. **Login** and **Customer** are one-to-one; everything else fans out from **Customer**.

— Who can read & write what

Database security is enforced at the row level — a customer logged in as Aanya can only see Aanya's rows, not anyone else's.

TABLE	CUSTOMERS CAN READ	CUSTOMERS CAN WRITE
Customer	Their own row	Their own row (name, dorm, allergens, ...)
Subscription	Their own rows	Their own rows (skip / pause / resume / change date)
Order	Their own rows	Never. Only Dormers writes (after Stripe confirms payment).
WhatsApp verification codes	Never	Never. Server-only.

9

Every banner, badge & modal

Beyond the main pages, Dormers has 17 conditional UI surfaces — banners, badges, and modal dialogs that pop up when a specific condition is met. Reference list:

SURFACE	WHEN IT APPEARS
"Up next" queued banner	Customer has a current plan AND a renewal queued behind it
Day-1 gold ribbon	First day of a new plan, before any meals delivered
Last-day renew banner	Today is the final day of the cycle and no renewal is queued
"Finish your profile" banner	Required profile field is blank — blocks all "Pick a plan" CTAs
Locked veg-days strip	Religious Mix customer with an active plan — read-only chips showing the chosen veg days
Webhook-delay spinner	After paying, before the new plan appears on the dashboard (under 20 seconds)
"Almost there..." fallback	Same as above but more than 20 seconds — adds a WhatsApp escape hatch
Success overlay	Briefly after a successful checkout — the celebration moment
Order-confirmation banner	Persistent confirmation right after checkout (sits at the top of the dashboard)
Skip confirm modal	After clicking "Skip tonight's meal", before it actually skips
Pause confirm modal	After clicking "Pause my plan", before it actually pauses
Change start date modal	From My plan, on a Waiting-to-start plan, when "Change start date" hasn't been used
Change email modal	From Profile → Security & verification → "Change email"
Change password modal	From Profile → Security & verification → "Change password"
WhatsApp verify modal	From Profile → Security & verification → "Verify" or "Re-verify"
"Pause unavailable today" tooltip	When the customer is in Skipped today state — pause button greys out
"Past 2 PM" skip lockout	After 2 PM Dubai time on any delivery day — skip button greys out

A

Plain English ↔ code names

Everything in this document uses customer-facing terms. The engineering team and the database use shorter, more technical names. Cross-reference [here](#).

PLAIN ENGLISH	CODE NAME	WHERE IT LIVES
Active (currently delivering)	Active	subscriptions.status
On hold (paused)	Paused	subscriptions.status
Waiting to start (queued / scheduled)	Scheduled	subscriptions.status
Skipped today	Skipped	subscriptions.status
Finished (plan ended)	Ended	subscriptions.status
Customer	customer_id	auth.users.id = customers.id
Login email	email	auth.users.email
Customer ID short code (CID)	cid	customers.cid
Delivery week	week_type ('5DAYS' or '6DAYS')	customers.week_type + subscriptions.week_type
Veg days picked	veg_days (text array of day names)	subscriptions.veg_days
Meal preference	meal_preference_type	customers.meal_preference_type
Number of meals delivered so far	delivered_meals	subscriptions.delivered_meals
Meals customer paid for in total	total_meals	subscriptions.total_meals
Skips used this cycle	skipped_meals_count (counts events, not meals)	subscriptions.skipped_meals_count

PLAIN ENGLISH	CODE NAME	WHERE IT LIVES
Days paused this cycle	<code>paused_days</code>	<code>subscriptions.paused_days</code>
Pause already used	<code>has_paused_before</code>	<code>subscriptions.has_paused_before</code>
Start date already changed	<code>start_date_changed_at</code>	<code>subscriptions.start_date_changed_at</code>
Status update (00:05 Dubai)	<code>subscription_status_tick()</code>	Postgres scheduled job
Pause counter (00:10 Dubai)	<code>subscription_pause_tick()</code>	Postgres scheduled job
Delivery counter (20:00 Dubai)	<code>subscription_delivery_tick()</code>	Postgres scheduled job
End-date math	<code>compute_subscription_end_date()</code>	Postgres function + TypeScript twin in <code>src/lib/end-date.ts</code>
End-date trigger	<code>trg_subscriptions_recompute_end_date</code>	Postgres trigger on <code>subscriptions</code>
Renew → queue logic	Stripe webhook	<code>src/app/api/webhook/route.ts</code>
"Skip tonight's meal" button	<code>skipMeal()</code>	<code>src/app/dashboard/actions.ts</code>
"Pause my plan" button	<code>pauseSubscription()</code>	<code>src/app/dashboard/actions.ts</code>
"Resume plan" button	<code>resumeSubscription()</code>	<code>src/app/dashboard/actions.ts</code>

PLAIN ENGLISH	CODE NAME	WHERE IT LIVES
"Change start date" button	<code>changeStartDate()</code>	<code>src/app/dashboard/actions.ts</code>
"Change email" modal	<code>requestEmailChange()</code>	<code>src/app/dashboard/profile/security-actions.ts</code>
"Change password" modal	<code>changePassword()</code>	<code>src/app/dashboard/profile/security-actions.ts</code>
"Verify WhatsApp" modal	<code>markWhatsappVerified()</code>	<code>src/app/dashboard/profile/security-actions.ts</code>
Profile-completion gate	<code>missingProfileFields()</code>	<code>src/lib/profile-completion.ts</code>
Veg-day per-day decision	<code>vegDayNumbersFor()</code>	<code>src/lib/veg-day.ts</code>

B

Test accounts

Eighteen accounts are seeded in the database. Universal password: `TestPass123!`. Each one exercises a distinct branch of the diagrams in §4–§9.

EMAIL	WHAT IT COVERS
<code>fresh@dormers.test</code>	Pristine — profile complete, no plans (first-time empty state)
<code>incomplete@dormers.test</code>	Profile incomplete — "Finish your profile" banner blocks all CTAs
<code>emailunverified@dormers.test</code>	Email never confirmed — Profile shows "Unverified" tag, "Resend" button
<code>active@dormers.test</code>	Standard mid-cycle Active plan
<code>activemax@dormers.test</code>	Monthly Max — bar grows on first skip (2 meals per delivery)
<code>paused@dormers.test</code>	On hold — Resume becomes the primary CTA
<code>skipped@dormers.test</code>	Skipped today — Pause button disabled with tooltip
<code>scheduled@dormers.test</code>	Only a Waiting-to-start plan — change-date button enabled
<code>queued@dormers.test</code>	Active + queued renewal — "Up next" banner visible
<code>dayone@dormers.test</code>	Day 1 of new plan — gold "Your first dinner" ribbon
<code>lastday@dormers.test</code>	Today is end date · no queue — Last-day renew banner
<code>returning@dormers.test</code>	Multiple finished plans, no current — returning-customer empty state
<code>datechange@dormers.test</code>	Scheduled with date already changed — change-date button locked
<code>fiveday@dormers.test</code>	5-days-a-week customer — pricing UI shows lower meal counts
<code>trialcd@dormers.test</code>	Recent Trial — checkout rejects another Trial within 15 days
<code>religiousmix6day@dormers.test</code>	Religious Mix · 6 days — checkout veg-day picker caps at 5 of 6
<code>religiousmix5day@dormers.test</code>	Religious Mix · 5 days — checkout veg-day picker caps at 4 of 5

EMAIL

WHAT IT COVERS

religiousactive@dormers.test

Religious Mix Active · veg days = Tue, Thu, Sat — per-day mix on Dashboard + Menu

